

TD1

TD 1 : Algorithme Parallèle

Ex. 1: Écrire un algo parallèle qui calcule la somme de 2 tableaux A et B de taille n dans un tableau C.

-> Mémoire partagée

-> Autant de mémoire que nécessaire.

```
C = malloc(sizeof(int)*n);
```

```
Pour i = 0 à n-1 faire en parallèle*:
```

```
    C[i] <- A[i] + B[i]
```

```
finpour
```

Justification* : Les calculs sont distincts. D'une itération à une autre, on n'utilise pas de variable commune ou liée. Conditions de **Bernstein.

On a n unités de calculs

$P = n$

$T_1(n) = O(n)$

$T_p(n) = 1$

$S_p(n) = T_1/TP = n/1 = n$

$E_p(n) = SP(n) / P = n/n = 1$

$E_p(n) = \frac{T_1(n)}{W_p(n)} = \frac{S_p(n)}{p}$	$S_p(n) = \frac{T_1(n)}{T_p(n)}$	$W_p(n) = p \cdot T_p(n)$
---	----------------------------------	---------------------------

- Ok maintenant imaginons qu'on n'ait pas autant de threads qu'on veut, on ne peut qu'utiliser **p unités**, avec $p < n$.

Répartition des données (itérations) par blocs

Modifier l'algorithme pour utiliser P unités avec $P < n$ et une répartition par blocs

```
reste <- n%p
```

```
pour u allant de 0 à P-1 faire en parallèle:
```

```
    nbCases[u] <- floor(n/P) // Qté min de travail de toutes les unités**
```

```
    si u < reste alors // on traite les reste 1ères unités
```

```
        nbCases[u]++ // on ajoute 1
```

```
        deb[u] <- u*nbCases[u]
```

```
    sinon
```

```
        deb[u] <- u*nbCases[u] + reste
```

```
    fini
```

```
    pour i[u] de deb[u] à deb[u] + nbCases[u] - 1 faire: [Séquentiel ! partie locale]
```

```
        C[i[u]] <- A[i[u]] + B[i[u]]
```

```
    finpour
```

$T_p(n) = \lceil n/P \rceil$
 $S_p(n) = n / \lceil n/P \rceil \approx P$
 $E_p(n) \approx 1$

****ATTENTION : On met nbCases[u] pour avoir des variables nbCases locales à chaque unité. Sinon tout le monde essaie de modifier nbCases et le comportement devient non-déterministe.**

Modifier l'algorithme pour utiliser une répartition cyclique (comme si on donnait des cartes à chaque thread)

| -012012012012010201201201021021020120120120120-|
 | -__-|

****Niggolas's code**** *Soit j'ai cook soit je suis cooked*

```

div <- N/P
Pour u de 0 à div en parallèle
    si u*p >= N
        return
    C[u*p] = A[u*p] + B[u*p]
finpour
**UI**
  
```

Correction:

```

Pour u de 0 à P-1 faire en parallèle:
    i[u] <- u
    tant que u i[u] < n faire:
        C[i[u]] <- A[i[u]] + B[i[u]]
        i[u] += p
    fintantque
finpour
  
```

[NOTE] La version cyclique n'est pas idéale sur CPU car elle ne garantit pas une bonne utilisation du cache contrairement à la version par blocs. Par contre ça peut être bien plus intéressant sur GPU.

Exercice 2 : Réduction

Définition : Réduire plusieurs valeurs à une seule



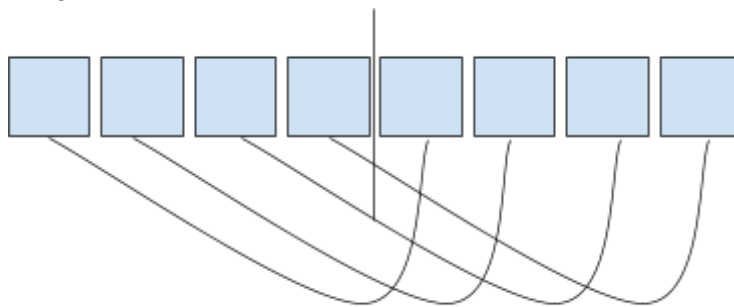
-> ne pas modifier le tableau T initial -> copie ($\log_2(n)$)

-> unités illimitées

*On peut d'une part faire deux par deux, et stocker le résultat sur le membre de gauche
Ou on peut prendre le premier et le dernier, puis le deuxième et l'avant-dernier, et ainsi de suite. L'avantage est que tous les résultats sont adjacents, à la moitié gauche. Puis on refait le processus jusqu'à avoir une seule case.*

Problème : Accès mémoire. On va au début, puis à la fin, puis au début, etc. On parcourt une grosse distance, et d'autres parcourent une petite. Il faut un accès régulier.

A la place on va faire ça :



Cela fonctionne si on a un tableau pair. Si c'est impair, il faut le garder pour l'étape suivante pour l'additionner à la fin sûrement

Algo :

pour i allant de 0 à n-1 *faire en parallèle:*

tmp[i] <- T[i] // On fait la copie en parallèle comme des bonhommes

finpour

nb <- n

tant que nb > 1 *faire en séquentiel* : //étapes de réduction

r <- nb%2 //élément central éventuel

nb <- floor(nb/2) // 1re moitié du tableau, on exclut l'élément central s'il y en a un
(floor est équivalent à une division entière, efficace)

pour u allant de 0 à nb-1 *faire en parallèle:*

tmp[u] += tmp[u+nb+r]

nb += r // Mise à jour pour étape suivante

finpour

fintantque

retour tmp[0]

p = O(n)

Tn(n) = log₂(n)

Sn(n) = n/log₂(n)

En(n) = 1/log₂(n)

Coût mémoire : $Mn(n) = \Theta(n)$

Principe de Brent (nombre d'unités optimal sur un algo existant) : $E = S/P \Rightarrow E = 1 \Rightarrow P = S/E = S$

Réécrire l'algorithme adapté à S unités ($=n/\log_2(n)$).

-> chaque unité calcule la somme partielle d'un bout du tableau -> $\log_2(n)$ données à traiter par unité.

-> répartition par blocs (moins simple à écrire) ou cyclique (plus simple)

[C'est bien pour l'utilisation mémoire aussi, car on a besoin de stocker uniquement S variable (1 par somme partielle, donc 1 par unité) et donc pas besoin de copier tout le tableau.]

$P \leftarrow n/\log(n)$

pour u allant de 0 à P-1 *faire en parallèle*:

$i[u] \leftarrow u$

$s[u] \leftarrow 0$

 tant que $i[u] < n$ faire:

$s[u] += T[i[u]]$

$i[u] += P$

 fintantque

finpour

$rés \leftarrow \text{réductionSimple}(s,P)$

$Tp(n) = \log_2(n) + \log_2(n/\log_2(n)) = [2^{\text{ème}} \text{ partie à cause de la réduction}] = O(\log_2(n))$

$Mp(n) = \Theta(n/\log_2(n))$

Double avantage : On économise de la mémoire, et on a un temps pareil, avec moins d'unités, donc plus d'efficacité.

Ex 3: Algo qui calcule les sommes par lignes d'un tableau 2D de taille $m \times n$ dans sL

ligne(T,l)

col(T,L)

pour l de 0 à N-1 *faire en parallèle (*m)*

$sL[l] \leftarrow B(+, \text{ligne}(\text{tab}, l), n) (n/\log_2(n))$

finpour

$T1(m,n) = m \times n$

$P = m \times n / \log_2(n)$

$Tp(m,n) = \log_2(n)$

$Sp(m,n) = m \times n / \log_2(n)$

$Ep(m,n) = Sp/p = 1$

Ajout du calcul des colonnes aussi, pas que des lignes

```

pour g de 0 à 1 faire en parallèle (2 gr)
  si g == 0 alors
    pour l de 0 à m-1 faire en parallèle (m)
      sL[l] <- B(+,ligne(tab,l), n) (n/log2(n))
    finpour
  sinon
    pour c de 0 à N-1 faire en parallèle (n)
      sC[c] <- B(+,col(tab,c), m) (m/log2(m))
    finpour
finpour

```

(Une partie fait les lignes, l'autre les colonnes)

$P = mn/\log(n) + mn/\log(m) = mn(\log(m)+\log(n))/\log(n)\log(m)$
 $Tp(m,n) = \max(\log(n), \log(m))$ (vu que c'est en parallèle, on prend le plus grand)
 $Sp(m,n) = 2*m*n / \max(\log(n), \log(m))$
 $Ep(m,n) = Sp / p = 2*\log(n)\log(m) / \log(m)+\log(m)*\max(...)$
 $= 2*\min(\log(n),\log(m)) / \log(m)+\log(m)$
 $= 1$ si $m = n$ et < 1 sinon

Variante :

```

pour u de 0 à m+n-1 faire en parallèle
  si u < m alors
    sL[u] <- B(+,...)
  sinon
    c <- u-m
    sC[c] <- B(+,...)

```

Exercice 4 : Traitement d'une image 1000*1000 niveaux de gris 1 pixel = 1 byte

Etapes :

- Lecture : 1 octet à la fois
- Réduction : 100*100 avec coût par pixel = 2000
- Intégration : 8 opération/pixel entiers
- Sauvegarde : 1 octet à la fois

	1	2	3	4
Degré	1	100*100	100*100	1
Coût élémentaire	1000*1000	2000	8	4*100*100

4 car on passe d'**octet** à **entier**

$T1 = 1 * 1000*1000 + 100*100 * 2000 + 8 * 100*100 + 4 * 100 * 100 = 21\ 120\ 000$
 $Tinf$ (temps parallèle minimum) = Somme des coûts élémentaires = 1 042 008

Sinf $\approx$ 20,27

$T_{10} = \text{arrondisup}(1/10) * 1000 * 1000 + \text{arrondisup}(100 * 100 / 10) * 2000 + 8 * \text{arrondisup}(100 * 100 / 10) + \text{arrondisup}(1/10) * 4 * 100 * 100 = 3048000$

$S_{10} = 6,93$

$E_{10} = 0,69$

Trouver le plus grand P possible pour avoir $E_p \geq .5$

$E_p = 1 / (1 + (p-1)R_s)$

R_s = ratio de séquentialité dans le temps total

$= (10^6 + 4 * 10^4) / (21\ 120\ 000) = 0.05$

On veut du 0.5, pas du 0.05 🤖

$\frac{1}{2} = 1 / (1 + (P-1)R_s)$

$p = 1 / R_s + 1 \approx 21$

$E_{21} = 0,503$

$E_{22} = 0,491$

Parallélisme de tâches

On doit calculer n valeurs de la fonction $f(x) = f_1(x) * f_2(x) + f_3(x)$

avec les coûts **10 20 30**

-> Les accès arithmétiques et mémoire coûtent 1

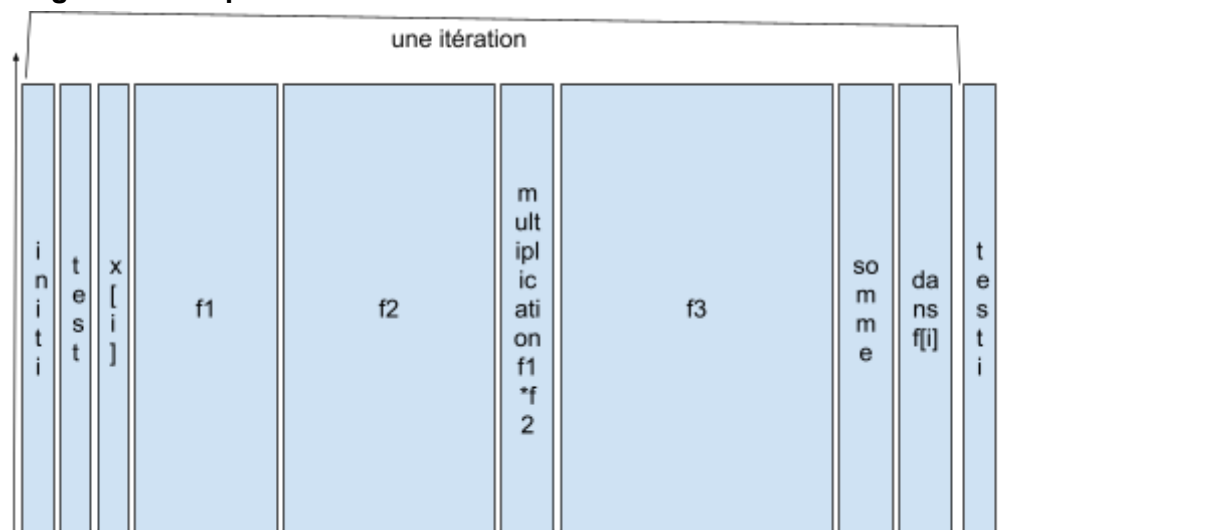
Algo séquentiel:

pour i de 0 à N-1 faire

$f[i] = f_1(x[i]) * f_2(x[i]) + f_3(x[i])$

finpour

Diagramme temporel :



Une itération = **66**

$T_1(n) = 1 + 66n$

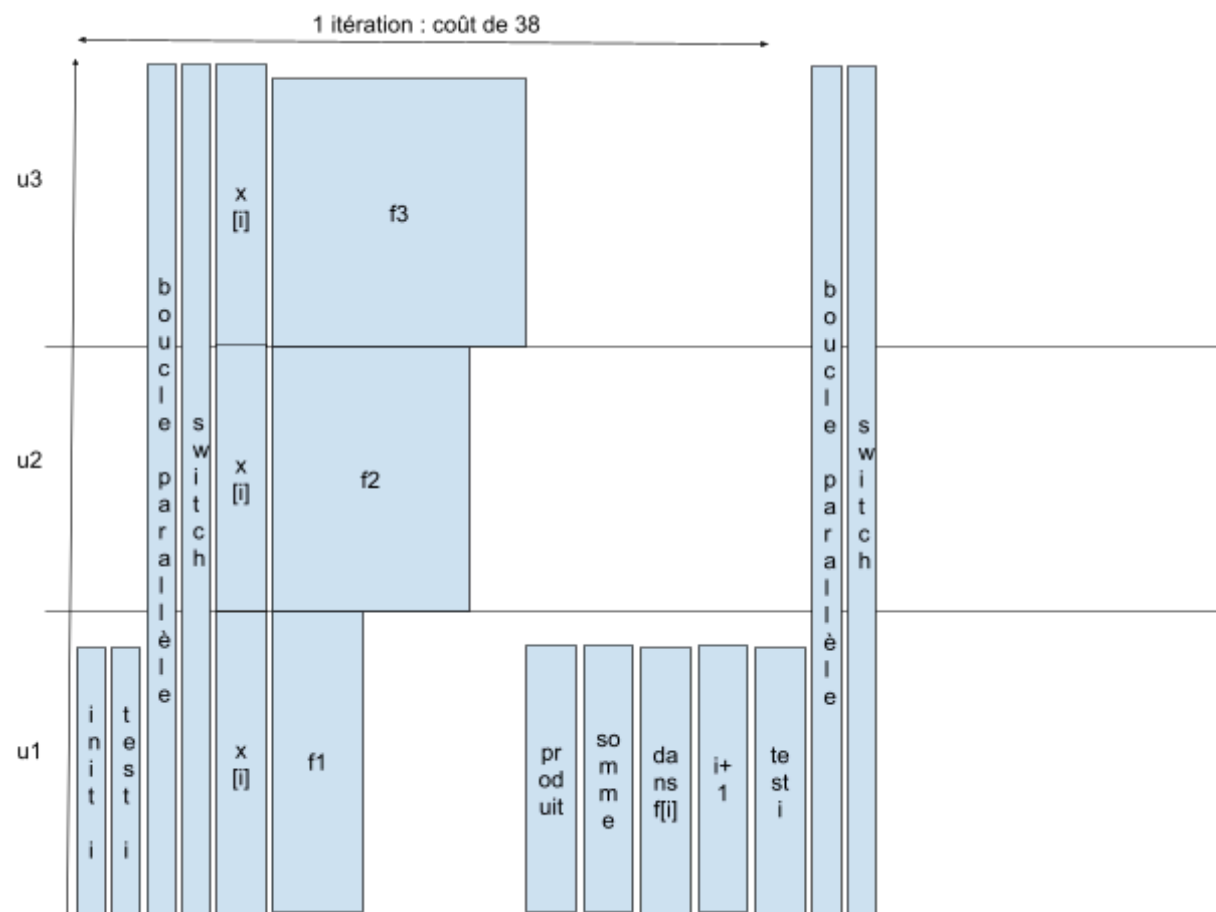
Maintenant on fait un algo en parallélisme de tâches avec 3 unités

- activation des unités, procédure de mise en place de boucle parallèle = coût de 1

```

pour i de 0 à N-1 faire
    pour u de 1 à 3 faire en parallèle
        switch u :
            case 1: res1 <- f1(x[i])
            case 2 : res2 <- f2(x[i])
            case 3 : res3 <- f3(x[i])
        finswitch
    finpour
    f[i] = res1*res2+res3
finpour
    
```

Diagramme temporel :



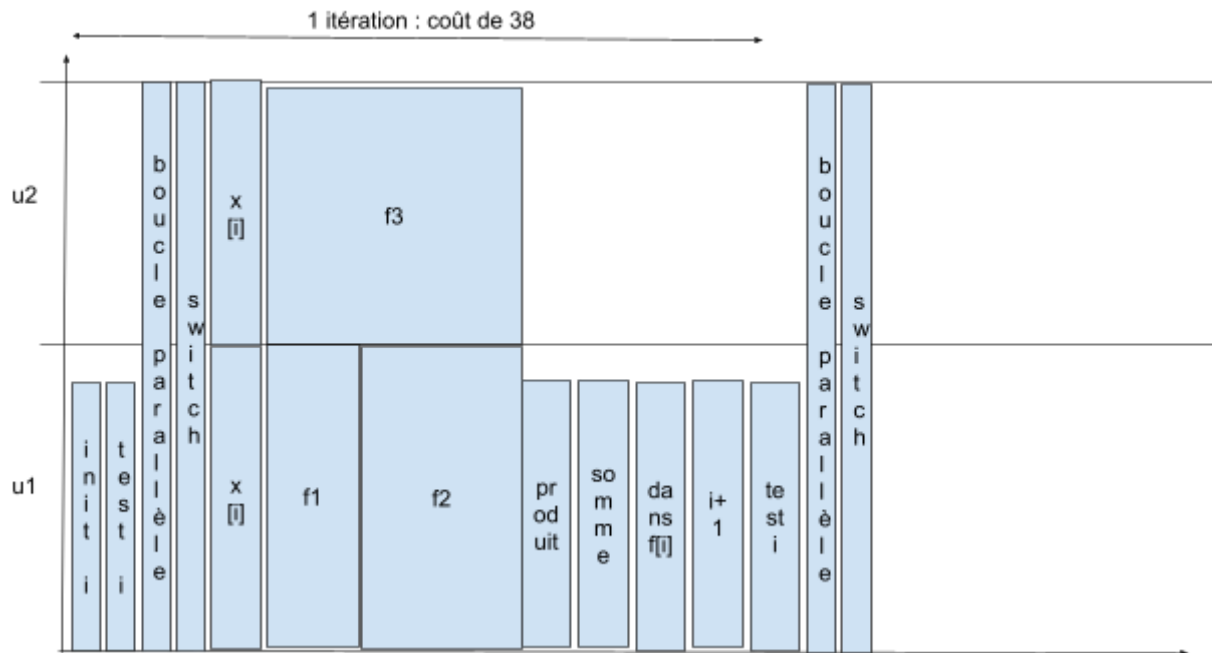
$$T3 = 1 + 38n$$

$$S3 = 1 + 66n / 1 + 38n \approx 66/38 = 1,74$$

$$E3 = 1,74/3 = 0,58$$

Efficacité pas folle, y a pas mal de trous

Cependant, on peut réduire pour avoir deux unités seulement. Regardez :



```

pour i de 0 à N-1 faire
  pour u de 1 à 3 faire en parallèle
    switch u :
      case 1: res1 <- f1(x[i]) * f2(x[i])
      case 2 : res2 <- f3(x[i])
    finswitch
  finpour
  f[i] = res1+res2
finpour

```

$$T2 = 1 + 38n$$

$$S2 = 1+66n / 1+38n \approx 66/38 = 1,74$$

$$E2 = 1,74/2 = 0,87$$

Now we're talking 😎

TD1 OpenMP

TD Machine

```
turbo-giga-salif@Odin telecom/S8/AP via C v15.2.1-gcc via Δ v4.2.3
⌘ gcc -fopenmp -Wall -lm ex_parallel.c -o parallel

turbo-giga-salif@Odin telecom/S8/AP via C v15.2.1-gcc via Δ v4.2.3
⌘ ./parallel
Sequential part
Process 7 among 8
Process 3 among 8
Process 6 among 8
Process 5 among 8
Process 1 among 8
Process 0 among 8
Process 4 among 8
Process 2 among 8
Return to sequential part.

turbo-giga-salif@Odin telecom/S8/AP via C v15.2.1-gcc via Δ v4.2.3
⌘ gcc -fopenmp -Wall -lm ex_parallel.c -o parallel

turbo-giga-salif@Odin telecom/S8/AP via C v15.2.1-gcc via Δ v4.2.3
⌘ ./parallel
Sequential part
Process 0 among 8
Process 4 among 8
Process 1 among 8
Process 3 among 8
Process 6 among 8
Process 2 among 8
Process 5 among 8
Process 7 among 8
Return to sequential part.
```

1er -> sans set OMP mais mal compilé (donc en fait c'est pareil qu'avec)

2e -> avec set OMP

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3 took 2s
⌘ ./ex_parallel
Sequential part
Process 0 among 1
Return to sequential part.
```

-> sans set OMP (bien compilé cette fois)

En mettant le set(OMP) avec l'argument "NO", lors de la compilation avec make, les arguments pragma omp parallel sont ignorés, alors que si l'on met "YES", ils sont pris en compte.

```
[ 50%] Linking C executable ex_nesting
[ 50%] Built target ex_nesting
[ 50%] Building C object CMakeFiles/ex_occurrences.dir/ex_occurrences.c.o
/home/nicolas03/Documents/TN/AlgoParallele/TP1/ex_occurrences.c: In function 'text_generation':
/home/nicolas03/Documents/TN/AlgoParallele/TP1/ex_occurrences.c:138: warning: ignoring '#pragma omp parallel' [-Wunknown-pragmas]
138 | #pragma omp parallel for
    |
[ 66%] Linking C executable ex_occurrences
[ 66%] Built target ex_occurrences
[ 75%] Building C object CMakeFiles/ex_parallel.dir/ex_parallel.c.o
/home/nicolas03/Documents/TN/AlgoParallele/TP1/ex_parallel.c: In function 'main':
/home/nicolas03/Documents/TN/AlgoParallele/TP1/ex_parallel.c:29: warning: ignoring '#pragma omp parallel' [-Wunknown-pragmas]
29 | #pragma omp parallel
    |
```

Avec 12 threads :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3 took 2s
⌘ OMP_NUM_THREADS=12 ./ex_parallel
Sequential part
Process 1 among 12
Process 3 among 12
Process 8 among 12
Process 11 among 12
Process 6 among 12
Process 7 among 12
Process 4 among 12
Process 2 among 12
Process 5 among 12
Process 9 among 12
Process 10 among 12
Process 0 among 12
Return to sequential part.
```

Puis avec 3 mais spécifié dans le code source :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
⌘ ./ex_parallel
Sequential part
Process 2 among 3
Process 0 among 3
Process 1 among 3
Return to sequential part.
```

```
#pragma omp parallel num_threads(3)
```

On remarque que la clause dans le code source a la priorité sur celle à l'exécution :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✂ OMP_NUM_THREADS=12 ./ex_parallel
Sequential part
Process 0 among 3
Process 1 among 3
Process 2 among 3
Return to sequential part.
```

Exercice 2 : Ex ex_loop

a)

Processus 0

Thread 0 executes iteration 0

Thread 0 executes iteration 1

Thread 0 executes iteration 2

Thread 0 executes iteration 3

Thread 0 executes iteration 4

Processus 2

Thread 2 executes iteration 10

Thread 2 executes iteration 11

Thread 2 executes iteration 12

Thread 2 executes iteration 13

Thread 2 executes iteration 14

Processus 3

Thread 3 executes iteration 15

Thread 3 executes iteration 16

Thread 3 executes iteration 17

Thread 3 executes iteration 18

Thread 3 executes iteration 19

Processus 1

Thread 1 executes iteration 5

Thread 1 executes iteration 6

Thread 1 executes iteration 7

Thread 1 executes iteration 8

Thread 1 executes iteration 9

Sum = 20 / 0.000489s

Nous voyons que les threads se répartissent les tâches dans la boucle. On a 4 threads et chacun prend 5 entrées dans la boucle.

b)

Processus 0

Thread 0 executes iteration 0
 Thread 0 executes iteration 1
 Thread 0 executes iteration 8
 Processus 1
 Thread 1 executes iteration 2
 Thread 1 executes iteration 3
 Thread 1 executes iteration 10
 Thread 1 executes iteration 11
 Thread 1 executes iteration 18
 Thread 1 executes iteration 19
 Processus 3
 Thread 3 executes iteration 6
 Thread 3 executes iteration 7
 Processus 2
 Thread 2 executes iteration 4
 Thread 2 executes iteration 5
 Thread 2 executes iteration 12
 Thread 2 executes iteration 13
 Thread 0 executes iteration 9
 Thread 0 executes iteration 16
 Thread 0 executes iteration 17
 Thread 3 executes iteration 14
 Thread 3 executes iteration 15
 Sum = 20 / 0.000379s

Nous voyons qu'avec l'argument `schedule(static, 2)`, il y a une répartition différente des processus. Sans cela, les threads ont un nombre égal d'itérations, mais avec cela ils ont un nombre variable. Cet exemple n'est pas représentatif car en l'exécutant à nouveau la configuration change. L'argument 2 est le chunk-size, donc chaque thread a une size de chunk allouée.

Lien de l'explication : <https://610yilingliu.github.io/2020/07/15/ScheduleinOpenMP/>

Ah en fait juste ça fonctionne en blocs de 2 itérations de chaque (4,5 ; 12,13, etc). C'était aussi simple que ça 🤖

- c) Si l'on exécute plusieurs fois le programme, les temps d'exécution sont fortement différents. Ils varient beaucoup d'une exécution à une autre.
- d) Avec l'ajout du `pragma` au-dessus de `sum++`, l'exécution est en général plus longue. avec `atomic`, cela fait que ça se lance en séquentiel
- e) Le temps est plus stable et un peu plus bas.

Exercice 3 : Imbalanced loop

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✂ ./ex_balancing
Proc 3 finishes at 8.909927s
Proc 0 finishes at 8.909939s
Proc 1 finishes at 8.909969s
Proc 2 finishes at 8.910063s
```

Avec le `nowait` :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✂ ./ex_balancing
Proc 0 finishes at 1.307581s
Proc 1 finishes at 3.849581s
Proc 2 finishes at 6.369774s
Proc 3 finishes at 8.894376s
```

```
#pragma omp for nowait
```

On voit les process finir les uns après les autres, avec des temps d'exécution bien différents puisqu'ils ne s'attendent pas

Avec `schedule(dynamic)` après le `nowait` :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✂ ./ex_balancing
Proc 2 finishes at 5.042399s
Proc 1 finishes at 5.149807s
Proc 3 finishes at 5.158702s
Proc 0 finishes at 5.186592s
```

Les temps sont à peu près équivalents. En lisant la doc d'openMP, on remarque que le `schedule(dynamic)` découpe la boucle en blocs de même taille, et les distribue à chaque thread (chaque thread prend le bloc en haut de la file d'attente et l'exécute), d'où les temps d'exécutions similaires.

En changeant dynamic pour guided 4 : (pour static c'est à peu près les mêmes temps)

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✧ ./ex_balancing
Proc 1 finishes at 4.931225s
Proc 0 finishes at 5.111540s
Proc 3 finishes at 5.171846s
Proc 2 finishes at 5.405897s
```

Dynamic : [dynamic](#) is an OpenMP [schedule](#) policy. The set of iterations is split in blocks of consecutive iterations called chunks, which are distributed to the threads in the team. Each thread executes a chunk, then requests another chunk, until no chunks remain to be distributed. The singularity of [dynamic](#) is that threads will synchronise behind to determine which thread is to pick the next chunk to process. This feature improves load balancing, at the cost of a thread coordination overhead. The optional argument passed to [schedule](#) tells how many iterations make a chunk, except [for](#) the chunk that contains the sequentially last iteration, which may have fewer iterations. If this optional argument is not present, the chunk size defaults to 1. Other OpenMP policies are [static](#), [guided](#), [auto](#) and [runtime](#).

Guided : [guided](#) is an OpenMP [schedule](#) policy. The set of iterations is split in blocks of consecutive iterations called chunks, which are distributed to the threads in the team. Each thread executes a chunk, then requests another chunk, until no chunks remain to be distributed. The singularity of [guided](#) is that the chunk size starts as large and gradually decreases, in order to progressively improve load balancing. These decreasing chunk sizes will eventually fall to 1 by [default](#), unless a minimum chunk size is passed via the optional argument (except the last chunk to assign, which may have fewer iterations that asked). Other OpenMP [schedule](#) policies are [static](#), [dynamic](#), [auto](#) and [runtime](#).

Static : [static](#) is an OpenMP [schedule](#) policy. It distributes the iteration chunks to the threads available following a round-robin distribution: first chunk to first thread, second chunk to second thread and so on. If no chunksize is passed by the user (see [schedule](#)), the chunk size is set so that every thread has at most 1 chunk. If the user passes a chunk size such that there are more chunks than threads, when the last thread receives a chunk, the distribution starts another dispatching round from the first thread. The distribution does as many rounds as needed to dispatch all chunks. Conversely, in the event that the user passes a chunk size such that there are more threads than chunks, only the first n threads will obtain a chunk, with n being the total number of chunks. Other OpenMP policies are [dynamic](#), [guided](#), [auto](#) and [runtime](#).

Exercice 4 : Nested parallelism

a :


```
Thread de niveau 1 : 0
  Thread de niveau 2 : 0 (0)
    Thread de niveau 3 : 0 (0 0)
Thread de niveau 1 : 1
  Thread de niveau 2 : 0 (1)
    Thread de niveau 3 : 0 (1 0)
```

Les nums 2 et 3 sont juste le thread 0.

b:

Thread de niveau 1 : 0

Thread de niveau 1 : 1

Thread de niveau 2 : 0 (0)

Thread de niveau 2 : 1 (0)

Thread de niveau 2 : 2 (0)

Thread de niveau 2 : 1 (1)

Thread de niveau 2 : 0 (1)

Thread de niveau 2 : 2 (1)

Thread de niveau 3 : 1 (1 1)

Thread de niveau 3 : 3 (1 1)

Thread de niveau 3 : 2 (0 2)

Thread de niveau 3 : 1 (0 2)

Thread de niveau 3 : 3 (0 2)

Thread de niveau 3 : 2 (1 1)

Thread de niveau 3 : 0 (1 1)

Thread de niveau 3 : 0 (0 2)

Thread de niveau 3 : 1 (1 2)

Thread de niveau 3 : 2 (1 2)

Thread de niveau 3 : 0 (1 2)

Thread de niveau 3 : 3 (1 2)

Thread de niveau 3 : 3 (0 0)

Thread de niveau 3 : 2 (0 0)

Thread de niveau 3 : 1 (0 0)

Thread de niveau 3 : 0 (0 0)

Thread de niveau 3 : 1 (1 0)

Thread de niveau 3 : 3 (1 0)

Thread de niveau 3 : 2 (1 0)

Thread de niveau 3 : 0 (1 0)

Thread de niveau 3 : 3 (0 1)

Thread de niveau 3 : 1 (0 1)

Thread de niveau 3 : 2 (0 1)

Thread de niveau 3 : 0 (0 1)

On a enfin les threads qui se répartissent la tâche

Exercice 5 : Generation of tasks

turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3 took 5s

⌘ ./ex_tasks

```
Thread 3 executes the work 0 in 0.202203s
Thread 2 executes the work 0 in 0.202240s
Thread 0 executes the work 0 in 0.202248s
Thread 1 executes the work 0 in 0.203523s
Thread 2 executes the work 1 in 0.404378s
Thread 0 executes the work 1 in 0.404399s
Thread 3 executes the work 1 in 0.404503s
Thread 1 executes the work 1 in 0.406101s
Thread 0 executes the work 2 in 0.606001s
Thread 2 executes the work 2 in 0.606242s
Thread 3 executes the work 2 in 0.606227s
Thread 1 executes the work 2 in 0.608911s
Thread 0 executes the work 3 in 0.809287s
Thread 3 executes the work 3 in 0.809039s
Thread 2 executes the work 3 in 0.810788s
Thread 1 executes the work 3 in 0.812136s
Thread 0 executes the work 4 in 1.011059s
Thread 2 executes the work 4 in 1.010815s
Thread 3 executes the work 4 in 1.012563s
Thread 1 executes the work 4 in 1.014804s
Thread 0 executes the work 5 in 1.213638s
Thread 3 executes the work 5 in 1.213171s
Thread 2 executes the work 5 in 1.214117s
Thread 1 executes the work 5 in 1.219161s
Thread 0 executes the work 6 in 1.414952s
Thread 3 executes the work 6 in 1.414766s
Thread 2 executes the work 6 in 1.414855s
Thread 1 executes the work 6 in 1.420667s
Thread 0 executes the work 7 in 1.617172s
Thread 3 executes the work 7 in 1.617092s
Thread 2 executes the work 7 in 1.616462s
Thread 1 executes the work 7 in 1.618394s
Thread 3 executes the work 8 in 1.823063s
Thread 0 executes the work 8 in 1.833699s
Thread 1 executes the work 8 in 1.825711s
Thread 2 executes the work 8 in 1.860998s
```

```
Thread 3 executes the work 8 in 1.823063s
Thread 0 executes the work 8 in 1.833699s
Thread 1 executes the work 8 in 1.825711s
Thread 2 executes the work 8 in 1.860998s
Thread 3 executes the work 9 in 2.021277s
Thread 0 executes the work 9 in 2.020809s
Thread 1 executes the work 9 in 2.026292s
Thread 2 executes the work 9 in 2.021982s
Thread 0 executes the work 10 in 2.227421s
Thread 1 executes the work 10 in 2.233952s
Thread 2 executes the work 10 in 2.231967s
Thread 3 executes the work 10 in 2.271704s
Thread 0 executes the work 11 in 2.424635s
Thread 2 executes the work 11 in 2.425323s
Thread 1 executes the work 11 in 2.430875s
Thread 3 executes the work 11 in 2.427130s
Thread 0 executes the work 12 in 2.628695s
Thread 1 executes the work 12 in 2.632629s
Thread 2 executes the work 12 in 2.635860s
Thread 3 executes the work 12 in 2.634433s
Thread 0 executes the work 13 in 2.830169s
Thread 2 executes the work 13 in 2.830436s
Thread 3 executes the work 13 in 2.830558s
Thread 1 executes the work 13 in 2.836110s
Thread 0 executes the work 14 in 3.030166s
Thread 2 executes the work 14 in 3.032446s
Thread 3 executes the work 14 in 3.032499s
Thread 1 executes the work 14 in 3.037450s
Thread 0 executes the work 15 in 3.232407s
Thread 2 executes the work 15 in 3.233664s
Thread 3 executes the work 15 in 3.233003s
Thread 1 executes the work 15 in 3.237754s
Thread 0 executes the work 16 in 3.435636s
Thread 3 executes the work 16 in 3.436439s
Thread 2 executes the work 16 in 3.438065s
Thread 1 executes the work 16 in 3.442925s
Thread 0 executes the work 17 in 3.636864s
Thread 3 executes the work 17 in 3.639611s
Thread 2 executes the work 17 in 3.638944s
```

```
Thread 2 executes the work 11 in 2.425323s
Thread 1 executes the work 11 in 2.430875s
Thread 3 executes the work 11 in 2.427130s
Thread 0 executes the work 12 in 2.628695s
Thread 1 executes the work 12 in 2.632629s
Thread 2 executes the work 12 in 2.635860s
Thread 3 executes the work 12 in 2.634433s
Thread 0 executes the work 13 in 2.830169s
Thread 2 executes the work 13 in 2.830436s
Thread 3 executes the work 13 in 2.830558s
Thread 1 executes the work 13 in 2.836110s
Thread 0 executes the work 14 in 3.030166s
Thread 2 executes the work 14 in 3.032446s
Thread 3 executes the work 14 in 3.032499s
Thread 1 executes the work 14 in 3.037450s
Thread 0 executes the work 15 in 3.232407s
Thread 2 executes the work 15 in 3.233664s
Thread 3 executes the work 15 in 3.233003s
Thread 1 executes the work 15 in 3.237754s
Thread 0 executes the work 16 in 3.435636s
Thread 3 executes the work 16 in 3.436439s
Thread 2 executes the work 16 in 3.438065s
Thread 1 executes the work 16 in 3.442925s
Thread 0 executes the work 17 in 3.636864s
Thread 3 executes the work 17 in 3.639611s
Thread 2 executes the work 17 in 3.638944s
Thread 1 executes the work 17 in 3.645330s
Thread 0 executes the work 18 in 3.843935s
Thread 2 executes the work 18 in 3.840249s
Thread 3 executes the work 18 in 3.841349s
Thread 1 executes the work 18 in 3.854682s
Thread 0 executes the work 19 in 4.042550s
Thread 2 executes the work 19 in 4.043508s
Thread 3 executes the work 19 in 4.043159s
Thread 1 executes the work 19 in 4.049628s
Computation finished in 42.557763s
```

Avec omp single :


```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
⌘ ./ex_tasks
Thread 0 executes the work 0 in 0.203024s
Thread 0 executes the work 1 in 0.403990s
Thread 0 executes the work 2 in 0.605605s
Thread 0 executes the work 3 in 0.807373s
Thread 0 executes the work 4 in 1.009803s
Thread 0 executes the work 5 in 1.211811s
Thread 0 executes the work 6 in 1.413687s
Thread 0 executes the work 7 in 1.615599s
Thread 0 executes the work 8 in 1.817418s
Thread 0 executes the work 9 in 2.019231s
Thread 0 executes the work 10 in 2.221470s
Thread 0 executes the work 11 in 2.423396s
Thread 0 executes the work 12 in 2.625116s
Thread 0 executes the work 13 in 2.826554s
Thread 0 executes the work 14 in 3.028206s
Thread 0 executes the work 15 in 3.231523s
Thread 0 executes the work 16 in 3.432879s
Thread 0 executes the work 17 in 3.634470s
Thread 0 executes the work 18 in 3.835796s
Thread 0 executes the work 19 in 4.038120s
Computation finished in 42.405608s
```

Maintenant chaque work est exécuté par un seul thread, il n'est pas mis en parallèle

D'après la doc : The single construct specifies that the associated structured block is executed by only one of the threads in the team (not necessarily the master thread), in the context of its implicit task. The other threads in the team, which do not execute the block, wait at an implicit barrier at the end of the single construct unless a `nowait` clause is specified.

Avec `#pragma omp task` avant chaque work :

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
```

```
⌘ ./ex_tasks
```

```
Thread 0 executes the work 0 in 0.202458s  
Thread 0 executes the work 1 in 0.404929s  
Thread 0 executes the work 2 in 0.607020s  
Thread 0 executes the work 3 in 0.808735s  
Thread 0 executes the work 4 in 1.011757s  
Thread 0 executes the work 5 in 1.212709s  
Thread 0 executes the work 6 in 1.416082s  
Thread 0 executes the work 7 in 1.616546s  
Thread 0 executes the work 8 in 1.819699s  
Thread 0 executes the work 9 in 2.020652s  
Thread 0 executes the work 10 in 2.223298s  
Thread 0 executes the work 11 in 2.423697s  
Thread 0 executes the work 12 in 2.626352s  
Thread 0 executes the work 13 in 2.827265s  
Thread 0 executes the work 14 in 3.028411s  
Thread 0 executes the work 15 in 3.230805s  
Thread 0 executes the work 16 in 3.433109s  
Thread 0 executes the work 17 in 3.635519s  
Thread 0 executes the work 18 in 3.837694s  
Thread 0 executes the work 19 in 4.042291s  
Computation finished in 42.429638s
```

Avec la directive task :


```
int main()
{
    double start, stop; // Timers to measure execution time
    double duration;    // Duration of the computation

    // Starting the clock
    start = omp_get_wtime();

    // Creation of threads
    #pragma omp single
    {
        int i;

        for(i=0; i<20; ++i){
            #pragma omp task
            {
                work(i);
            }
        }
    }

    // Stopping the clock
    stop = omp_get_wtime();

    // Duration of the computation
    duration = stop - start;

    // Display
    printf("Computation finished in %lfs\n", duration);

    // Return of success signal at the end of the program
    return(EXIT_SUCCESS);
}
```

```
turbo-giga-salif@Odin S8/AP/build via Δ v4.2.3
✂ ./ex_tasks
Thread 0 executes the work 0 in 0.202600s
Thread 0 executes the work 1 in 0.405114s
Thread 0 executes the work 2 in 0.610767s
Thread 0 executes the work 3 in 0.890284s
Thread 0 executes the work 4 in 1.031377s
Thread 0 executes the work 5 in 1.215764s
Thread 0 executes the work 6 in 1.415563s
Thread 0 executes the work 7 in 1.615798s
Thread 0 executes the work 8 in 1.817685s
Thread 0 executes the work 9 in 2.019682s
Thread 0 executes the work 10 in 2.221801s
Thread 0 executes the work 11 in 2.422861s
Thread 0 executes the work 12 in 2.625670s
Thread 0 executes the work 13 in 2.827492s
Thread 0 executes the work 14 in 3.029041s
Thread 0 executes the work 15 in 3.230274s
Thread 0 executes the work 16 in 3.432450s
Thread 0 executes the work 17 in 3.634496s
Thread 0 executes the work 18 in 3.843474s
Thread 0 executes the work 19 in 4.038369s
Computation finished in 42.531155s
```

Y a pas vraiment de changement...

Exercice 6 : Application on a more concrete example

En séquentiel :

✂ ./ex_occurrences

Text size : 10000

Seed : 1773391104

Nb Threads : 1

Text generation...complete.

Seq time : 0.000259 s

Sequential and parallel results are IDENTICAL !

	Seq	Par	Frequencies
a =	205	205	
b =	201	201	
c =	212	212	
d =	211	211	
e =	263	263	
f =	183	183	
g =	144	144	
h =	206	206	
i =	202	202	
j =	239	239	
k =	192	192	
l =	173	173	
m =	200	200	
n =	244	244	
o =	228	228	
p =	229	229	
q =	169	169	
r =	234	234	
s =	223	223	
t =	142	142	
u =	188	188	
v =	220	220	
w =	223	223	
x =	186	186	
y =	208	208	
z =	231	231	

Parallel :

Time : 0.000240 s

Speedup : 1.079674

Efficiency : 1.079674

En parallèle :

✂ ./ex_occurrences

Text size : 10000

Seed : 1773391312

Nb Threads : 1

Text generation...complete.

Seq time : 0.000259 s

Sequential and parallel results are DIFFERENT :-/

	Seq	Par	Frequencies
a =	221	97	
b =	205	101	
c =	221	96	
d =	163	64	
e =	160	75	
f =	229	81	
g =	127	73	
h =	242	102	
i =	281	98	
j =	203	150	
k =	262	179	
l =	200	85	
m =	232	84	
n =	204	100	
o =	259	130	
p =	156	75	
q =	219	113	
r =	192	98	
s =	290	106	
t =	257	177	
u =	244	107	
v =	219	85	
w =	183	68	
x =	201	154	
y =	139	56	
z =	217	87	

Parallel :

Time : 0.000099 s

Speedup : 2.621386

Efficiency : 2.621386

```

void par_counting(int nbThreads, char *text, int size, int *occs)
{
    int i;

    //
    // PART TO MODIFY !
    //
    #pragma omp parallel
    {
        // Occurrences initialisation
        memset(occs, 0, NB_LETTERS * sizeof(int));

        // Text parsing
        #pragma omp for nowait
        for(i=0; i<size; ++i){
            if(text[i] >= 'a' && text[i] <= 'z'){
                occs[text[i]-'a']++;
            }
            if(text[i] >= 'A' && text[i] <= 'Z'){
                occs[text[i]-'A']++;
            }
        }
    }
}

```

C'est bien mais... Les résultats sont faux :(

Une des façons d'avoir les bons résultats et d'avoir le schedule(dynamic).

TD2 OpenMPI

TD2 Machine

Exercice 1:

Question a (avec MPI)

```
⌘ mpirun -n 1 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 1

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 2 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 2
I am process 1 among 2

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 3 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 3
I am process 1 among 3
I am process 2 among 3

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 3 among 4
I am process 1 among 4
I am process 2 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 8 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 8
I am process 1 among 8
I am process 5 among 8
I am process 3 among 8
I am process 7 among 8
I am process 4 among 8
I am process 6 among 8
I am process 2 among 8
```

Question b (sans MPI)

```
turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ ./creation
Activation of additional MPI debugging information
I am process 0 among 1
```

Question c


```

⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 2 among 4
I am process 1 among 4
I am process 3 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 3 among 4
I am process 2 among 4
I am process 1 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 3 among 4
I am process 2 among 4
I am process 1 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 1 among 4
I am process 3 among 4
I am process 2 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
⌘ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 3 among 4
I am process 1 among 4
I am process 0 among 4
I am process 2 among 4

```

Well akschually the display order is random and can't be predicted. That is why we have different process outputs. 🤓👉

For sure 😎

Question d :

Après modification du code, c'est dans le bon ordre :

```
turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ ./creation
Activation of additional MPI debugging information
I am process 0 among 1
>>>>>>>>> MPI ERROR on process 0 : /home/turbo-giga-salif/.datif/telecom/S8/AP/TP_OpenMPI/creation.c main:45 <<<<<<<<<
[Odin:00000] *** An error occurred in MPI_Ssend
[Odin:00000] *** reported by process [1008140288,0]
[Odin:00000] *** on communicator MPI_COMM_WORLD
[Odin:00000] *** MPI_ERR_RANK: invalid rank
[Odin:00000] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[Odin:00000] *** and MPI will try to terminate your MPI job as well)

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 1 among 4
I am process 2 among 4
I am process 3 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 1 among 4
I am process 2 among 4
I am process 3 among 4

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 4 -host localhost:8 --bind-to core:overload-allowed ./creation
Activation of additional MPI debugging information
I am process 0 among 4
I am process 1 among 4
I am process 2 among 4
I am process 3 among 4
```

Exercise 2

```
Value of pi : 3.141592653552712
Difference with standard value : 3.708e-11
Computed in : 0.141297s

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 2 -host localhost:8 --bind-to core:overload-allowed ./pi
Activation of additional MPI debugging information
Process 1 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 0 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Value of pi : 3.141592653552712
Difference with standard value : 3.708e-11
Computed in : 0.147130s

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 5 -host localhost:8 --bind-to core:overload-allowed ./pi
Activation of additional MPI debugging information
Process 4 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 3 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 2 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 1 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 0 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Value of pi : 3.141592653552712
Difference with standard value : 3.708e-11
Computed in : 0.238385s

turbo-giga-salif@Odin AP/TP_OpenMPI/build
$ mpirun -n 8 -host localhost:8 --bind-to core:overload-allowed ./pi
Activation of additional MPI debugging information
Process 2 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 3 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 1 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 6 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 7 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 4 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 5 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Process 0 computes 10000000 trapezoids of width 1.000000e-07 from x = 0.000000 with 1 thread(s) and obtains 3.141592653552712
Value of pi : 3.141592653552712
Difference with standard value : 3.708e-11
Computed in : 0.248341s
```

On remarque que les temps d'exécution changent un peu (+ de threads = + long), mais la précision non

